Day 129 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 4 reference

Watch: The Reparameterization Trick (ML & DL Explained) | The Reparameterisation Trick | Variational Inference (Machine Learning and AI Academy)

`day129_vae_reparameterization.py`, segment by segment, in the order `main()` uses things. The trick, the KL term, the VAE loss and its training, and the two gradient estimators are new; Day 126's network parts are imported unchanged and do the forward and backward passes. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: mu, logvar = the encoder's two halves · eps = the N(0, 1) draw · z = the code · enc, dec = encoder and decoder layer lists · rec, kl = the two parts of the loss · beta = the KL weight

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day126_autoencoders as d126m
```

- `import day126_exercise as d126` — Day 126's forward, backward, recon_loss, init_net and Adam do all the network arithmetic.
- `with contextlib.redirect_stdout(io.StringIO()):` — keeps the imported exercise's own checks out of today's output.

Segment 1 — The trick

reparam and kl_normal

MATH

$$z = \mu + e^{\frac{1}{2}\log \sigma^2}\,\varepsilon; \quad KL = \tfrac{1}{2}\,\Sigma_j\,(\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2)$$

CODE

```
def reparam(mu, logvar, rng):
    eps = rng.standard_normal(mu.shape).astype(mu.dtype)
    return mu + np.exp(0.5 * logvar) * eps, eps
def kl_normal(mu, logvar):
    return 0.5 * (np.exp(logvar) + mu ** 2 - 1 - logvar).sum(1)
```

- `rng.standard_normal(mu.shape).astype(mu.dtype)` — one N(0, 1) draw per code number, in the same precision as the network.
- `np.exp(0.5 * logvar)` — σ from the log-variance: always positive.
- `.sum(1)` — the KL of independent dimensions is the sum of each dimension's KL.

vae_loss: forward, then back through the draw

MATH

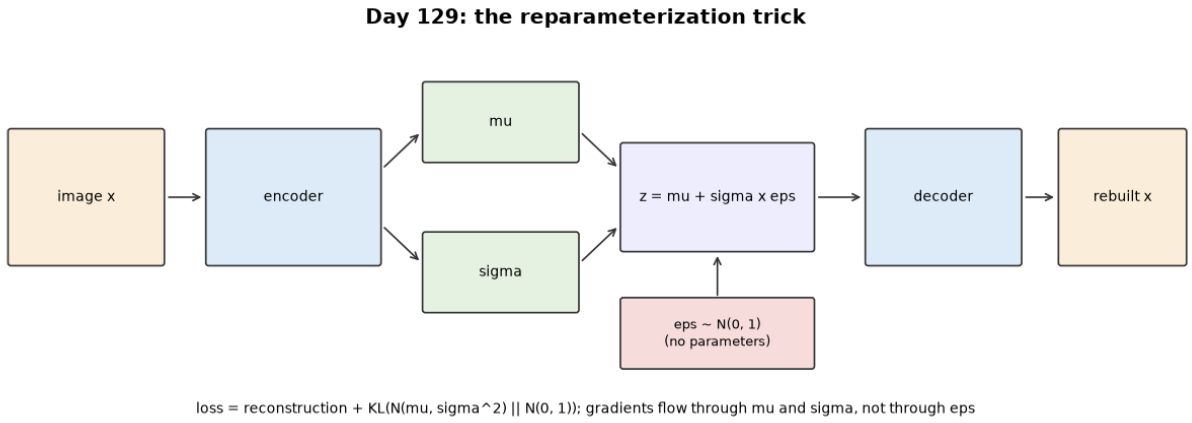
$$\partial L/\partial \mu = \partial L/\partial z + \beta\mu/n; \quad \partial L/\partial logvar = \partial L/\partial z \cdot \varepsilon\sigma/2 + \beta(\sigma^2 - 1)/2n$$

CODE

```
def vae_loss(enc, dec, X, rng, beta=1.0, acts_e=("relu", "linear"), acts_d=("relu", "linear"),
    eps=None):
    n = len(X); H = d126.forward(enc, X, list(acts_e)); out = H[-1]; k = out.shape[1] // 2
    mu, logvar = out[:, :k], out[:, k:]
    if eps is None:
        z, eps = reparam(mu, logvar, rng)
    else:
        z = mu + np.exp(0.5 * logvar) * eps
    A = d126.forward(dec, z, list(acts_d)); rec, dL = d126.recon_loss(A[-1], X, "bce")
    kl = float(kl_normal(mu, logvar).mean())
    gdec, dz = d126.backward(dec, A, list(acts_d), dL)
    dmu = dz + beta * mu / n
    dlogvar = dz * eps * 0.5 * np.exp(0.5 * logvar) + beta * 0.5 * (np.exp(logvar) - 1) / n
    genc, _ = d126.backward(enc, H, list(acts_e), np.hstack([dmu, dlogvar]))
    return rec + beta * kl, rec, kl, genc, gdec
```

- `out[:, :k], out[:, k:]` — the encoder's last layer split in two: the first k columns are means, the rest log-variances.
- `gdec, dz = d126.backward(dec, ...)` — Day 126's backward also returns the gradient at the decoder's input: dL/dz.
- `dz * eps * 0.5 * np.exp(0.5 * logvar)` — the chain rule through z = μ + σε with σ = exp(½ logvar).
- `np.hstack([dmu, dlogvar])` — the two halves' gradients joined back side by side, as the encoder's output was.

SYSTEM DIAGRAM



The reparameterization trick.

train_vae

CODE

```
def train_vae(X, k, hidden=256, epochs=10, batch=128, lr=1e-3, beta=1.0, seed=0, Xval=None):
    enc = d126.init_net([X.shape[1], hidden, 2 * k], seed); dec = d126.init_net([k, hidden,
        X.shape[1]], seed + 1)
    oe, od = d126.Adam(enc, lr), d126.Adam(dec, lr); rng = np.random.RandomState(seed + 2); hist =
        []
    for _ in range(epochs):
        o = rng.permutation(len(X))
        for s in range(0, len(X), batch):
            _, _, _, ge, gd = vae_loss(enc, dec, X[o[s:s + batch]], rng, beta)
            oe.step(enc, ge); od.step(dec, gd)
        ev = X[:2000] if Xval is None else Xval
        hist.append(vae_loss(enc, dec, ev, np.random.RandomState(99), beta)[:3])
    return enc, dec, hist
```

- `d126.init_net([X.shape[1], hidden, 2 * k], seed)` — the encoder's last layer has 2k outputs: k means and k log-variances.
- `oe.step(enc, ge); od.step(dec, gd)` — two Adam optimisers, one for each half.
- `np.random.RandomState(99)` — a fixed draw for the held-out loss, so epochs are compared on the same ε.

Segment 2 — Two estimators

score_grad and reparam_grad

MATH

$$\partial/\partial\theta\, E_q[f(z)] = E_q[f(z)\, \partial \log q(z)/\partial\theta] = E_\varepsilon[f'(\mu + \sigma\varepsilon)\, \partial z/\partial\theta]$$

CODE

```
def score_grad(f, mu, sigma, rng, n):
    z = mu + sigma * rng.standard_normal(n)
    return f(z) * (z - mu) / sigma ** 2, f(z) * ((z - mu) ** 2 / sigma ** 3 - 1 / sigma)
def reparam_grad(df, mu, sigma, rng, n):
    eps = rng.standard_normal(n); z = mu + sigma * eps
    return df(z), df(z) * eps
```

- `f(z) * (z - mu) / sigma ** 2` — the score function: f's value times the slope of log q in μ.
- `df(z), df(z) * eps` — the trick: f's slope, times dz/dμ = 1 and dz/dσ = ε.

Segment 3 — The experiments

demo_fit: a run that may blow up

CODE

```
with np.errstate(all="ignore"):
    for t in range(2000):
        s = np.exp(ls)
        gm, gs = (score_grad(f, mu, s, rng, 1) if est == "score function" else reparam_grad(df,
            mu, s, rng, 1))
        gm, gs = gm[0] + mu, gs[0] * s + (s * s - 1)
        mu -= lr * gm; ls -= lr * gs; path.append((mu, np.exp(ls)))
```

- `np.errstate(all="ignore")` — silences overflow warnings inside the block: blown-up runs are counted afterwards instead.
- `ls = log sigma` — learning log σ keeps σ positive; the σ gradient is multiplied by σ to convert.

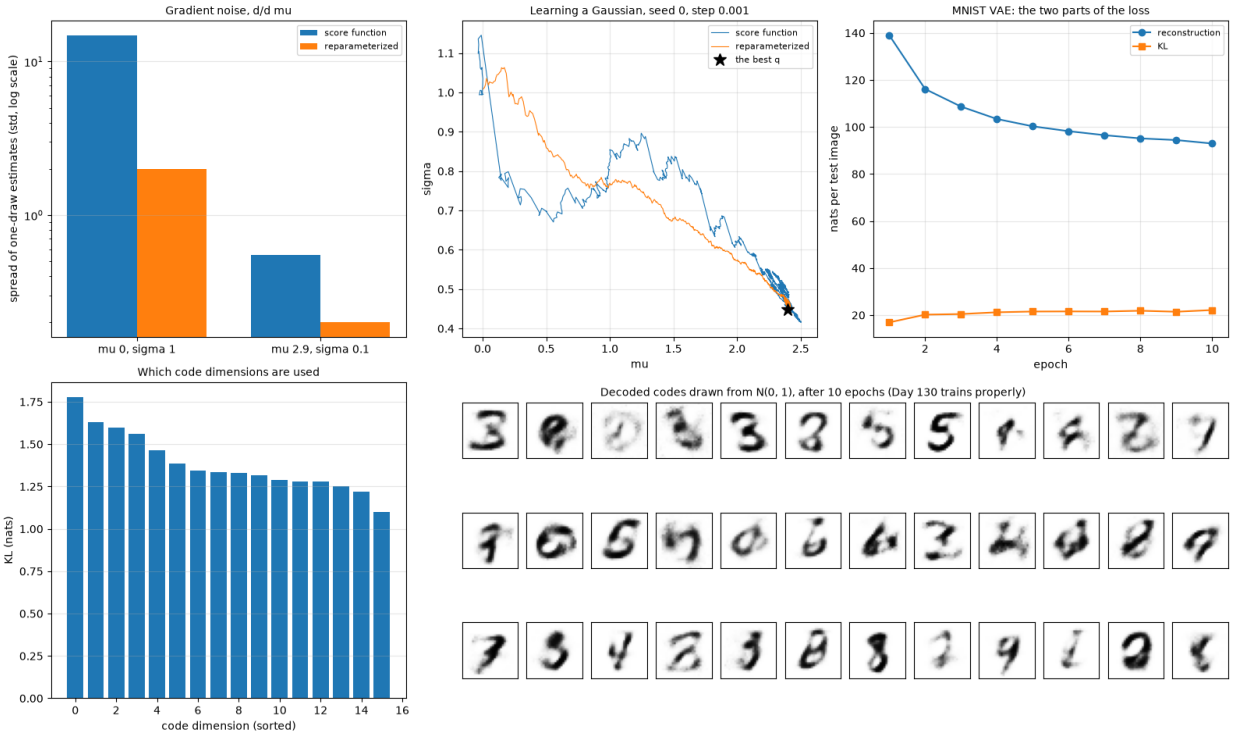
Counting the runs that blew up

CODE

```
F = np.array(final); ok = np.all(np.isfinite(F), 1); err = np.abs(F[ok] - target)
```

- `np.isfinite(F)` — False for inf and nan: the marks of a run that blew up.
- `int((~ok).sum())` — ~ flips True and False; the sum counts the failed runs.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `rng.standard_normal(shape)` — draws from N(0, 1).
- `out[:, :k], out[:, k:]` — split columns in two.
- `np.hstack([a, b])` — join side by side.
- `np.errstate(all="ignore")` — silence floating-point warnings in a block.
- `np.isfinite(a)` — True where a is a normal number.
- `1_000_000` — underscores make long numbers readable.

End of Day 129 syntax reference. For the experiments and what they showed, see the Day 129 complete guide.